PORTFOLIO

Yiyou Xu

Monash University · Computer Science (Advanced Computer Science) · Class of 2027
yxuu0276@student.monash.edu · github.com/RGXeva

01
Distributed EV Charging Network
C · Open MPI · OpenMP · Cluster

Alert round trip **0.81 → 0.32 ms**

02
Parallel Prime Search
C · MPI · OpenMP · Cluster

16-process speed-up **11.4× → 15.6×**

03
Flippy Bit Web Game
TypeScript · RxJS · Vitest

87 tests, **21/21** injected faults caught

04
MiniZinc-Lite Parser & Solver
Haskell · Parser combinators

Full BNF grammar, **30 + 22** test models

05
Quant Research Toolchains
Rust · Python · LEAN

Solo since **Dec 2025**, live trading

06
GPU Image Rotation
CUDA · Numba

5.0× faster end to end at 2048^2

01 Distributed EV Charging Network Simulation

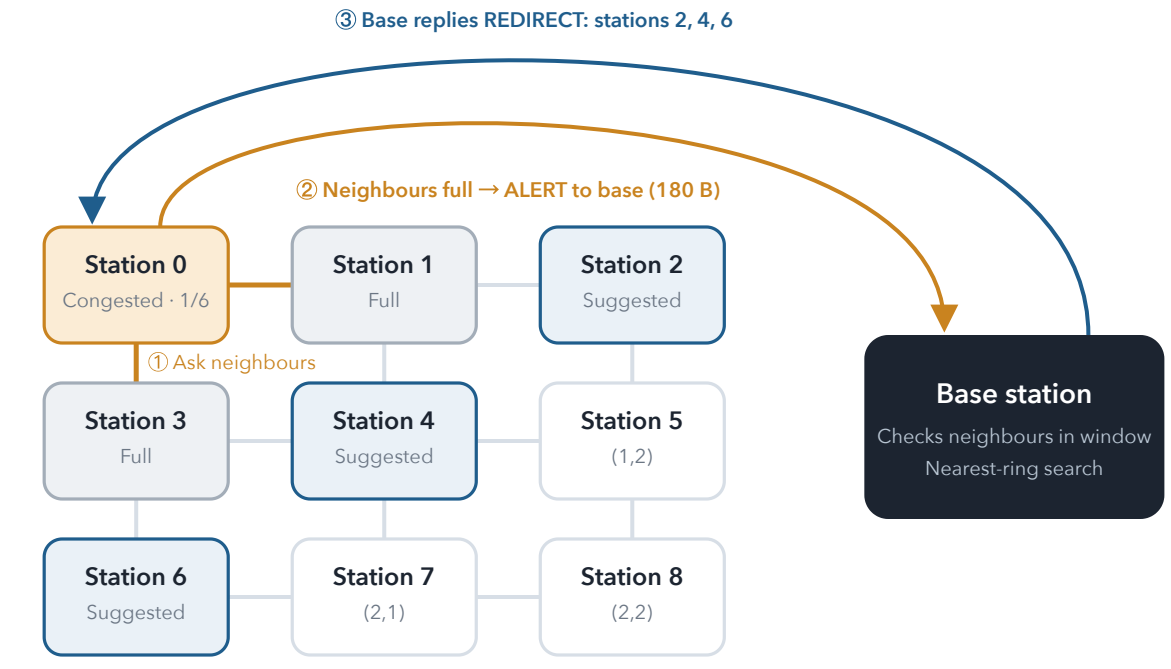
Two-person course project · Lead, about 90% of the work · Sep – Oct 2026 · Code github.com/RGXeva/ev-charging-sim

C | Open MPI | OpenMP | SLURM | Python

A grid of charging stations simulated as a distributed sensor network. A congested station asks its neighbours first; only when they are full too does it ask the base station for the nearest free one.

WHAT I BUILT

- Compared **8 network topologies** and chose a 2-D grid: one MPI process per station, with neighbours found through a Cartesian topology
- One OpenMP thread per charging port, each writing only its own slot: **no locks** and no data races
- Designed a **non-blocking, on-demand** neighbour protocol; persistent receives answer neighbours at any time, so it cannot deadlock
- The base station checks neighbour reports within a time window, then searches **Manhattan rings** outwards and returns every free station at the nearest distance
- Graceful shutdown: the base broadcasts TERMINATE, nodes finish open work and exit together, then free threads, communicators and memory
- Owned the architecture, about 1,500 lines of C, cluster experiments, slides and the presentation



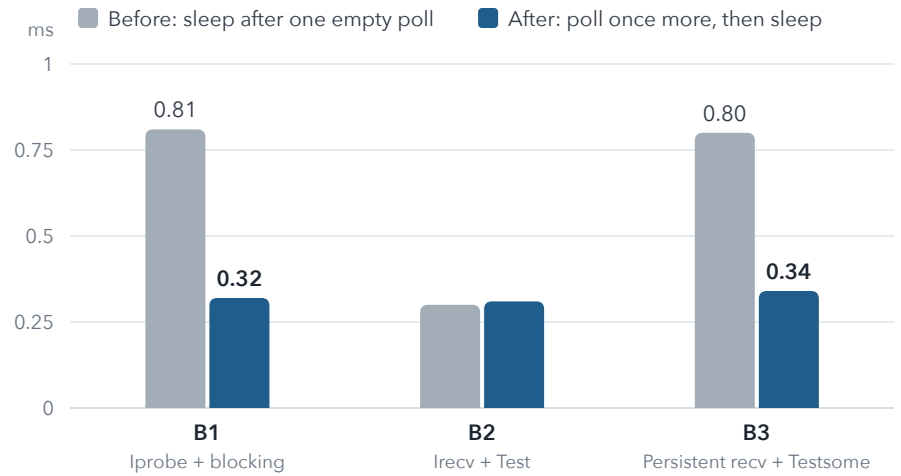
01 Choose by measurement, debug with data

Distributed EV Charging Network Simulation · Experiments and quality

CAAS cluster | ASan | UBSan | Python analysis

A polling gap, found and fixed: alert latency down about 60%

Median alert round trip (ms) · 6×6 grid, 37 processes, 3 hosts



Sleeping after a single empty poll misses messages that land during the poll. Polling once more before sleeping cut B1 and B3 latency by more than half; B2's call re-checks on its own, so it was already fast.

Three node designs; N2 chosen by measurement

Node design	Messages per node per cycle	Shutdown
N1 · blocking Sendrecv every cycle	3.39	277 ms
N2 · non-blocking, on demand (chosen)	**1.97**	**84 ms**
N3 · Neighbor_allgather every cycle	3.39	279 ms

32/32
automated tests pass on the cluster

0
errors in our code from ASan, UBSan and LeakSanitizer

400+
experiment runs, local and cluster (up to 5 nodes × 16 cores)

Live run on Monash's CAAS cluster · 3×3 grid + base station

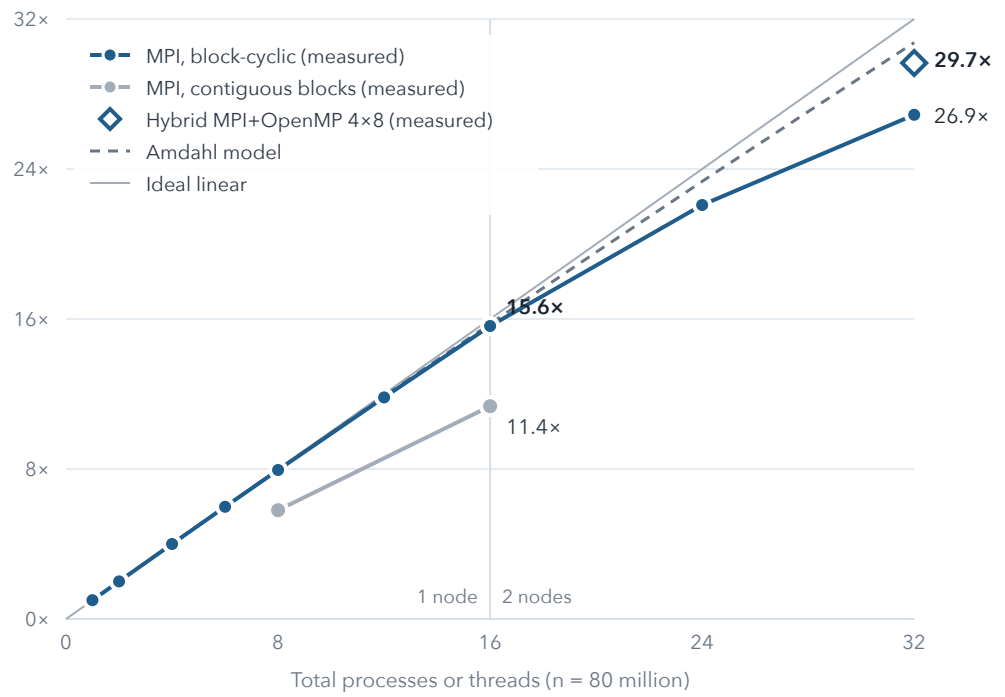
```
$ bash scripts/demo_caas.sh
srun: job 197921 has been allocated resources
[base t=  2.500s] alert #1 node 8 (2,2) free 1/6 -> suggest nearest available: 2(0,2) 4(1,1)
  6(2,0)
[base t=  6.500s] alert #2 node 6 (2,0) free 1/6 -> no available node nearby
[base t=  7.501s] alert #1 node 2 (0,2) free 0/6 -> suggest nearest available: 0(0,0)
  ...
[base] 3x3 N2/B2: 20.10 s, 23 alerts (20 candidates, 3 none nearby), 101 local redirects, 725
  messages
[base] neighbour exchange median 0.274 ms | alert round trip median 0.267 ms | shutdown
  96.497 ms
```

02 Parallel prime search: find the bottleneck, then fix it

Two-person course project · Lead, about 90% of the work · Aug – Sep 2026 C MPI OpenMP Pthreads SLURM

Speed-up versus number of workers

Median cluster runs against the serial version; dashed line: Amdahl model from the measured serial fraction



Starting from a serial version, I built Pthreads, OpenMP, MPI and hybrid MPI+OpenMP versions, measuring before every optimisation.

Bottleneck 1: load imbalance

Larger numbers need more trial divisions, so contiguous blocks overloaded the last process. Dealing blocks of 4,096 numbers round-robin moved the slowest-to-mean ratio **1.41 → 1.00** and 16-process speed-up **11.4× → 15.6×**.

Bottleneck 2: serial output on rank 0

A pilot run showed all output formatting piled on rank 0. Having each rank format its own primes, with rank 0 only writing the file, moved pilot speed-up **12.2× → 15.1×**.

873
timed runs across 245 configurations

100%
of outputs match an independent sieve

0.16%
measured serial fraction; model within 1% on one node

03 Flippy Bit: a functional reactive web game

Individual course assignment · Aug – Sep 2026 · Play it at rgxeva.github.io/flippy-bit TypeScript RxJS Vite Vitest



Hexadecimal numbers fall toward a line; before each one lands, the player flips 8 binary digits to match its value.

ALL STATE LIVES IN ONE STREAM

Five event streams clock, keys, mouse, pause, restart → **One action type** each event maps to an Action → **merge + scan** folded into one game state → **render()** the only side effect

Every game rule is a pure function that never touches the DOM or reads the clock, so tests can drive hundreds of time steps directly in Node.

TESTING THE TESTS WITH MUTATIONS

87 tests over the pure modules and event streams

21/21 deliberately injected faults caught by the tests

+2 tests added to close the blind spot mutation found

Faults included judging the wrong target, removing the speed cap, reversing bit order and mutating state in place. Mutating a target's height in place survived the first round because the immutability test only checked the opening state, so I added a frame-by-frame snapshot check.

04 MiniZinc-Lite: constraint model parser and solver

Individual course assignment · Oct – Nov 2026 Haskell Parser combinators BNF TypeScript front end

Reads a constraint-modelling language, parses it into a syntax tree, checks names and scope, then finds every feasible solution or the optimal one.

Model text
.mzn source

→

Parser combinators
one parser per BNF rule

→

Semantic checks
undeclared or duplicate names the grammar cannot express

→

Syntax tree
declarations, constraints, goal

→

Solve
all feasible solutions, or maximise and minimise

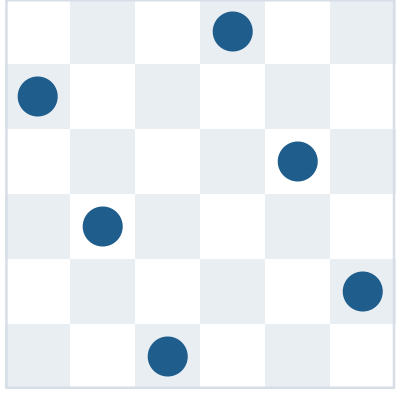
Input: the 6-queens problem

```
int: n = 6;
array[1..n] of var 1..n: q;

constraint allDifferent(q);
constraint allDifferent(i in 1..n)(q[i] + i);
constraint allDifferent(i in 1..n)(q[i] - i);

solve satisfy;
```

First solution found



```
[("q[1]",2),("q[2]",4),("q[3]",6),
 ("q[4]",1),("q[5]",3),("q[6]",5)]
… 4 solutions in total
```

30
sample models whose solutions the tests compare with the expected answers

22
invalid models the tests check the parser rejects

A web front end shows solutions live as the model is typed on the left, and saves them with one click.

05 Quant research toolchains

Personal projects · sole designer, developer and researcher · Dec 2025 – present

Rust | Python | QuantConnect LEAN | Docker | Tardis

Since December 2025 I have built two research toolchains on my own, and I have live-trading experience. The digital-asset toolchain started in Python and moved to Rust after benchmarking showed latency was the bottleneck. Only the engineering structure is shown here; strategies and returns stay private.

DIGITAL-ASSET RESEARCH TOOLCHAIN (PYTHON → RUST)

Archive
raw market data stored

→

Normalise
one format across sources

→

Replay
markets replayed in event time

→

Strategy and factors
factors mined and backtested on Tardis data

→

Execution and risk
simulated fills and risk checks

US EQUITY RESEARCH AND BACKTESTING (LEAN)

Data validation
market-data quality checks

→

Features
research features

→

Signals
strategy signals

→

Transaction costs
fees and other cost assumptions

→

Order interface
isolated from live-broker permissions

Engineering checks I focus on

Event ordering · quantity precision · fee assumptions · error handling · reproducibility. Requirements, test cases and code review are done with AI coding assistants, and every generated line is reviewed and tested by hand.

Factor research

Familiar with WorldQuant BRAIN and Qlib, with factor construction and backtest evaluation; factor mining on Tardis historical data.

06 GPU image rotation

Two-person course project · Oct 2026 CUDA Numba Python

Rotating images on the GPU: one thread per output pixel, each inverse-mapping back to the source image and writing only its own pixel, so no atomics are needed.

- 16 × 16 thread blocks with a ceiling-divided grid; threads past the edge exit at a bounds check
- Timing covers the whole copy-in, kernel and copy-out pipeline, compared against a compiled CPU version
- Every run is checked against the CPU result; all trials match
- The larger the image, the smaller the share of data transfer and the bigger the GPU's lead

End-to-end speed-up (including memory copies)

30° rotation versus single-core CPU, median of 5 runs

